



GROUP 2

Summoner's Rift - Transportation Web Application

Computational Thinking Project Proposal

November 2025

Abstract

Ho Chi Minh City's rapid urbanization presents critical challenges to its transportation infrastructure, characterized by a highly fragmented and congested mobility network. Commuters and residents are forced to manually synchronize the disparate rhythms of public bus schedules, and the overwhelming crescendo of private vehicles through different app. This lack of integration results in significant operational inefficiencies, including suboptimal routing, and chronic exposure to severe traffic congestion, which in turn impacts urban quality of life.

We propose the development of an integrated "HCMC Transportation" web platform, builed to unify these transit modalities. This system will provide users with a single point of access for route optimization, aggregating data from public bus systems, and predictive traffic models.

This document outlines a Proof of Concept (PoC). The PoC is designed to validate the core technical feasibility of data integration, user adoption hypotheses, and the platform's value proposition for safety and efficiency against rigorous, predefined performance metrics specific to the HCMC environment. Following a successful PoC, we project an agile 3-month, four-phase production schedule for full-scale deployment.

Contents

1	Overview	1
1.1	Problem Definition	1
1.2	Proposed Solution	1
1.2.1	PoC Objectives	1
1.2.2	Constraints	2
1.2.3	Success Metrics	2
1.3	Stakeholders	3
1.4	Decomposition	4
1.5	Team Roles Overview	5
2	Features	7
2.1	Routing	7
2.1.1	Introduction	7
2.1.2	Input & Output	7
2.1.3	Work Breakdown	7
2.1.4	Requirements	9
2.1.5	Risk Management / Testing Method	11
2.1.6	Timeline	11
2.2	Recommendation System	13
2.2.1	Introduction	13
2.2.2	Input & Output	13
2.2.3	Work Breakdown	14
2.2.4	Requirements	15
2.2.5	Timeline	16
2.3	Safety & Monitoring	16
2.3.1	Introduction	16
2.3.2	Input & Output	17
2.3.3	Work Breakdown	17
2.3.4	Requirements	20
2.3.5	Risk Management / Testing Method	21
2.3.6	Timeline	22
2.4	Camera Ingestion Pipeline	23
2.4.1	Introduction	23
2.4.2	Input & Output	23
2.4.3	Work Breakdown	24
2.4.4	Requirements	25
2.4.5	Risk Management / Testing Method	26
2.4.6	Timeline	27
2.5	Database	27
2.5.1	Introduction	27
2.5.2	Input & Output	28

2.5.3	Work Breakdown	29
2.5.4	Requirements	29
2.5.5	Risk Management / Testing Method	30
2.5.6	Timeline	31
2.6	UI/UX Platform	31
2.6.1	Introduction	31
2.6.2	Input & Output	31
2.6.3	Work Breakdown	32
2.6.4	Requirements	32
2.6.5	Risk Management / Testing Method	33
2.6.6	Timeline	34

Chapter 1

Overview

1.1 Problem Definition

Travelers, from leisure visitors to business professionals, face significant and costly friction in planning and executing urban itineraries, which stems from a core problem of lacking nuances and fragmented data. Critical decision-making data on bus transit schedules, live traffic, and operating hours exists in disconnected silos.

This forces users to manually compare multiple apps, leading to:

- Inefficient planning under complex constraints (time, budget, transfers).
- Suboptimal in-the-moment transport choices, failing to balance cost, ETA.
- Reactive, unactionable responses to congestion and crowding.

1.2 Proposed Solution

A web platform separating pre-trip routing from in-trip recommendations, with a safety/alerting layer.

- Routing (pre-trip): generates one itinerary (Shortest) under user constraints.
- Recommender (in-trip): live ETA/cost bus options with confidence scores in a kanban-style UI.
- Safety & alerts: context-aware warnings on congested segments.

In-scope transport modes: bus, taxi /motorbike, walking. Trains/flights are out-of-scope for MVP.

1.2.1 PoC Objectives

Success is defined by achieving statistically significant signals against predefined goals:

- **Feasibility:** $\geq 80\%$ of all generated test itineraries must successfully satisfy all hard constraints (time windows, budgets, mobility limits).

- **Adoption:** $\geq 60\%$ user acceptance of the system's top-3 generated daily plans.
- **In-Trip Value:** Achieve an on-time arrival rate of $\geq 80\%$ and a missed-connection rate of $\leq 5\%$.
- **Safety Value:** Deliver alert precision of $\geq 80\%$ and limit user exposure to congested segments to ≤ 15 minutes per day.
- **Qualitative:** Secure an average CSAT score of $\geq 4.2/5$ on both planning and in-trip guidance

1.2.2 Constraints

Geography (MVP): 1 city only (Ho Chi Minh City).

API availability (core recs endpoint): $\geq 99.0\%$.

Latency: p95 recommendation $\leq 1.5s$ (aligned with metric); p50 $\leq 400ms$.

1.2.3 Success Metrics

Planning (Before Trip)

- **Feasibility:** Pass if $\geq 90\%$ of generated itineraries satisfy all hard constraints (time windows, budgets, mobility limits).

$$\text{Feasibility}\% = \frac{\text{hard constraints satisfied}}{\text{hard constraints applicable}} \times 100$$

- **User Acceptance of Plans:** Pass if $\geq 60\%$ of users select one of the top-3 generated plans.

In-Trip Recommendations (Public Transport)

- **On-Time Arrival Rate:** Pass if $\geq 80\%$. For trip leg j with scheduled ETA_j and actual arrival A_j :

$$\text{OnTime}_j = \begin{cases} 1, & |A_j - ETA_j| \leq 10 \text{ min} \\ 0, & \text{otherwise} \end{cases} \quad \text{OnTime Rate}\% = \frac{\sum_j \text{OnTime}_j}{\text{number of requests}} \times 100$$

- **Missed-Connection Rate:** Pass if $\leq 5\%$.
- **Cost Prediction Error:** Pass if $\leq 12\%$ (average relative error between predicted and actual).

$$\text{Error}_i = \frac{|\text{ActualCost}_i - \max(\text{PredCost}_i)|}{\max(\text{PredCost}_i)} \times 100$$

Safety & Congestion

- **Alert Precision:** Pass if $\geq 80\%$ (alerts lead to measurable time/congestion benefit).
- **Exposure to Busy Segments:** Pass if ≤ 15 min/day per user.
- **User Safety Feedback (Qualitative):** Pass if average CSAT $\geq 4.2/5$.

Experience & Quality

- **App Stability (Crash-Free):** Pass if $\geq 99.5\%$ of sessions are crash-free.
- **Recommendation Latency (p95):** Pass if ≤ 1.5 s.

1.3 Stakeholders

Stakeholder	Role / Interest	Impact / Influence
Urban Commuters & Travelers	End users who will use the platform to plan, book, and manage trips.	High — their adoption, feedback, and satisfaction determine project success.
Project Team / Developers	Responsible for building, testing, and maintaining the platform.	High — directly control execution quality and timeline.
UX/UI Designers	Ensure intuitive and efficient user experience across pre-trip and in-trip modules.	Medium to High — influence user satisfaction and adoption.
Data Engineers / Integration Specialists	Handle APIs for transit, ride-hail, and e-bike data integration.	High — ensure reliable data flow and accuracy.
Product Manager / Project Lead	Oversees PoC and production phases; ensures milestones and metrics are met.	High — strategic decision-maker for progress and quality control.

1.4 Decomposition

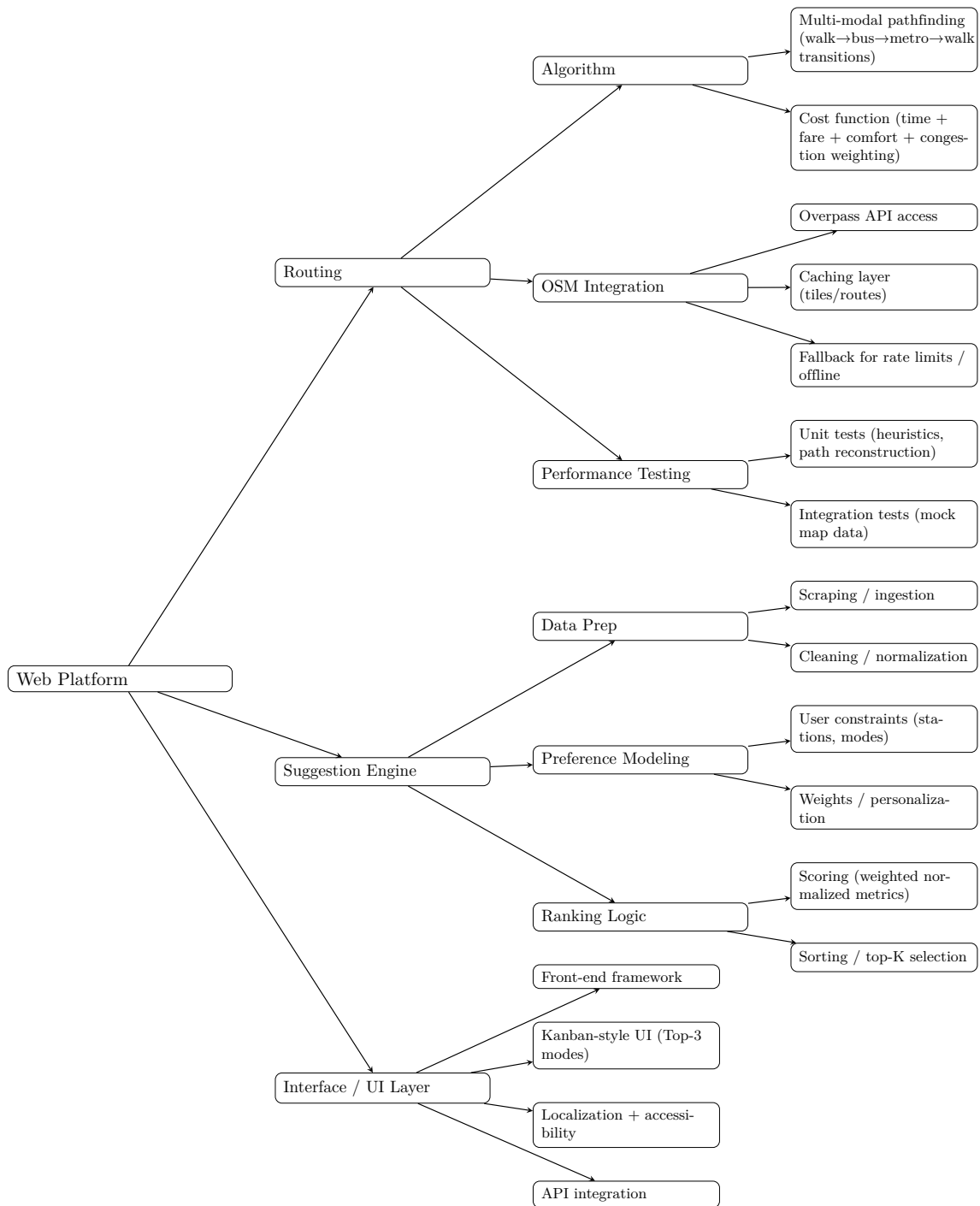


Figure 1.1: System decomposition overview — user-facing services (part 1 of 2)

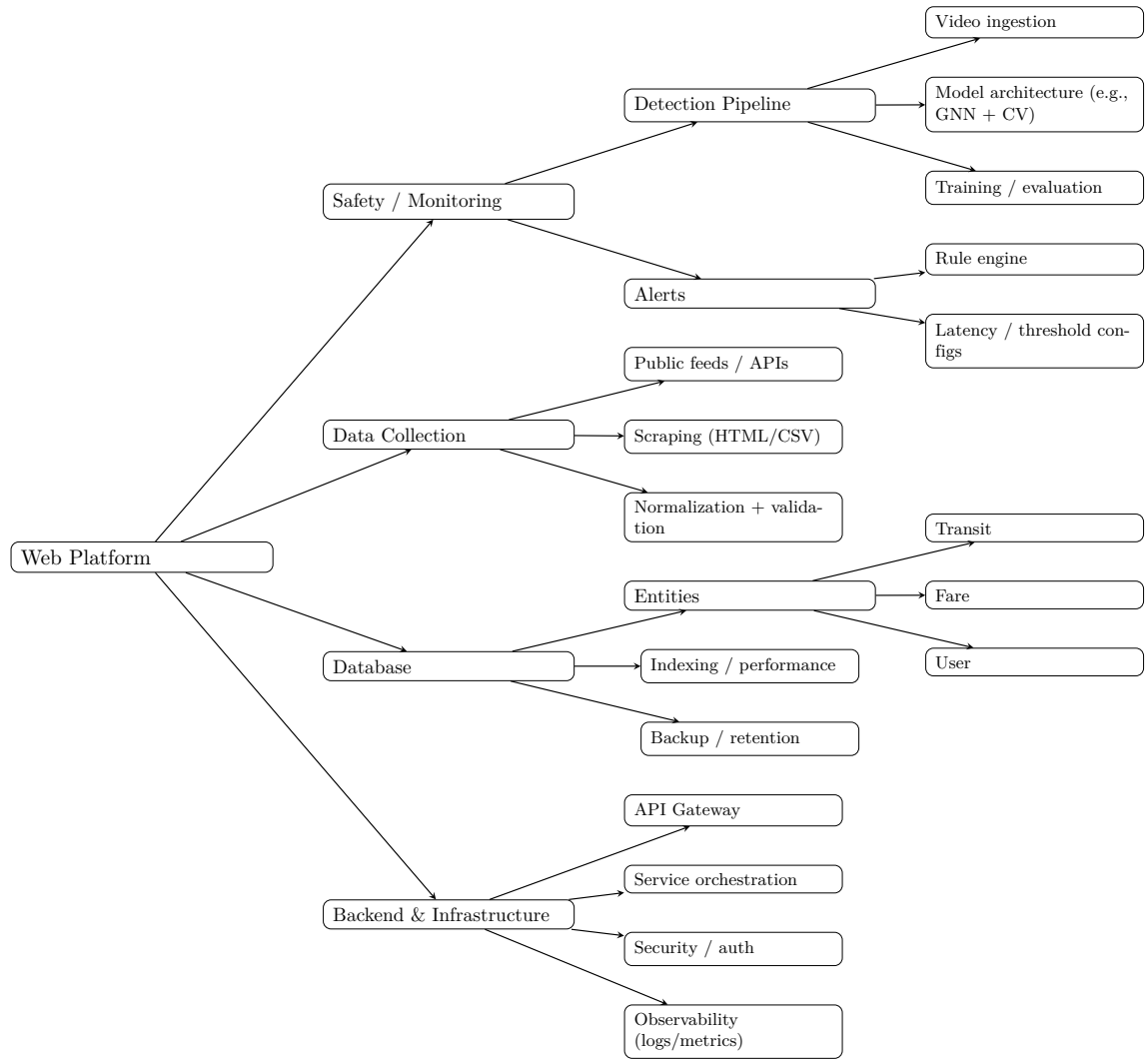


Figure 1.1: System decomposition overview — data and platform foundations (part 2 of 2)

1.5 Team Roles Overview

Summarize responsibilities for each core role:

- **Lead:** coordination, scope control, cross-area decisions, quality gates.
- **Routing:** algorithms, OSM/data integration, performance validation.
- **Recommendation:** data prep, preference modeling, ranking logic and evaluation.
- **Safety/Monitoring:** detection pipeline, alert rules, privacy and latency safeguards.
- **Data/Database:** ingestion pipelines, schemas, retention/backup, performance tuning.
- **UI/UX & Integration:** front-end flows, accessibility/localization, API wiring, user feedback collection.

Detailed Assignment Table

Role	Member	Key Responsibility
Routing	Nguyễn Thành Đạt (ID: 24127021)	Algorithm design, TomTom API integration, performance tuning
Recommendation System	Hoàng Trung Hiếu (ID: 24127041)	Suggestion logic, ranking metrics, feedback-driven tuning
Lead & Congestion	Lương Hưng Phát (ID: 24127298)	Real-time congestion detection, alert rule calibration
Data Collection	Nguyễn Minh Khoa (ID: 24127188)	Scrape real-time feeds, normalization, validation checks
Database	Nguyễn Anh Khôi (ID: 24127430)	Schema design, indexing, backup & retention strategy
UI/UX & Integration	Nguyễn Tấn Hiệu (ID: 24127373)	Frontend implementation, cross-module API integration

Table 1.1: Team roles and ownership

Chapter 2

Features

2.1 Routing

2.1.1 Introduction

The Routing Module constitutes the foundational architecture of the "Transportation Platform." Its primary objective is to resolve a critical inefficiency within Ho Chi Minh City's transit ecosystem: the current lack of a unified location search and multi-modal route calculation capability. This module is designed to process and deliver optimized, dynamic routes across all relevant transit modalities, including private vehicles, bus, and pedestrian options.

To ensure adherence to the accelerated 11-week development schedule, and, critically, to leverage the robust real-time traffic data necessary for meaningful route optimization, the module will be developed via the integration of TomTom's Search and Routing APIs.

Architecturally, this module will operate as a decoupled backend service (internal API), exposing three primary functional endpoints for consumption by other platform components and team members.

- **Geocoding/Search:** Convert text addresses into coordinates and vice versa.
- **Routing:** Calculate the optimal route (time, distance, list of coordinates) between two points.
- **Map Generation:** Automatically use data from the Routing step to create and export an interactive .html map file (using Folium), with the route, start point, and end point already drawn.

2.1.2 Input & Output

Inputs: origin, destination, time, preferences.

Outputs: candidate routes, costs, and explanations.

2.1.3 Work Breakdown

Tasks are divided into 3 main functional groups, corresponding to the user's activity flow:

Selecting Start and End Points (Search & Geocoding)

This is the step to get coordinates (Lat/Lon) from user input.

- **Build Endpoint POST /api/search:**
 - Receive address (text string) from the client.
 - Integrate search_api logic into the endpoint.
- **Call TomTom Search API:**
 - Handle general search logic (/search/address.json).
 - Handle nearby search logic (/nearbySearch/.json) if the client provides the current location's lat, lon.
- **Process results:**
 - Filter results not belonging to "Ho Chi Minh".
 - Standardize returned JSON: Only send necessary information to the UI (name, address, lat, lon).
- **Security:** Hide the TomTom API key (do not expose to the frontend).

Route Calculation (Routing)

This is the step to calculate the path after having 2 sets of coordinates.

- **Build Endpoint POST /api/route:**
 - Receive start_lat, start_lon, end_lat, end_lon from the client.
 - Receive options: travelMode (car, motorcycle...) and routeType (fastest, shortest...).
- **Integrate tomtom_route logic:**
 - Call TomTom calculateRoute API.
 - Pass the correct parameters (vehicle, route type, traffic=true).
- **Error Handling:**
 - Manage cases where TomTom doesn't find a route and returns 4xx, 5xx errors.
 - Return a 404 error code to the UI if no route is found ("routes" not in data).

Map Generation and Export

This is the final step and the main output of the Routing Module.

After collecting and processing all data from the TomTom API, the module will be responsible for aggregating and drawing this data into a standalone interactive .html map file.

This task uses logic from the visualize_path_on_map function in the demo code.

- **Aggregate input data:**
 - Receive `coords` (list of route coordinates).
 - Receive `start` (start point coordinates).
 - Receive `end` (end point coordinates).
- **Render the map (Using Folium):**
 - Initialize a `folium.Map` object and automatically fit the frame (`m.fit_bounds`) to the route.
 - Draw Markers: Add 2 Markers ("Start" and "End" icons) at the `start` and `end` points.
 - Draw the path (PolyLine): Draw the route on the map (including the border and the main line as in the demo code).
- **Export file (Main task):**
 - Call the `m.save()` function to save the entire map object (markers, path) as a single HTML file (e.g., `route_map.html`) on the server's disk.
- **Completion:**
 - When the `route_map.html` file has been successfully created and saved, the Routing Module's task is considered complete.
 - The API (Endpoint `/api/route`) will return a simple confirmation message, e.g., `"status": "success", "message": "Map created: route_map.html"`.

2.1.4 Requirements

Hardware

- Development Machine: 1 Personal Laptop (Windows, macOS, or Linux) with an internet connection for programming and API testing.
- Deployment Server: Cloud hosting service (such as Render) that supports Python backend applications.

This platform will host the FastAPI/Flask internal API and provide a public endpoint for integration with other modules.

Software

- **Language & Framework:**
 - Language: Python 3.x.
 - API Framework: FastAPI (recommended for high performance and automatic API documentation) or Flask (simple, easy to set up).
- **Main Python Libraries (Dependencies):**
 - requests: Essential library for sending HTTP requests (API calls) to the Tom-Tom server.

- folium: Key library to render GPS data (coordinates, paths) and create interactive .html map files.
- urllib: Use the urllib.parse submodule to encode URLs before calling the API.
- fastapi: Framework library for building the API.
- uvicorn: ASGI server, necessary to run the FastAPI application.
- python-dotenv: Used to manage the TomTom API key securely in an .env file.

- **Services & APIs Used (External Services):**

- TomTom Developer Account: Mandatory for registering and receiving an API Key.
- TomTom Search API: API used for the search function, converting addresses (strings) to coordinates (Lat/Lon).
- TomTom Nearby Search API: Used to search for locations within a 5km radius.
- TomTom Routing API: Core API, used to calculate routes, returns a summary (time, distance) and points (list of coordinates) to draw on the map.

- **Development Tools & Environment:**

- IDE/Editor: VS Code (Visual Studio Code).
- Version Control: Git and GitHub (to store code and collaborate).
- API Testing: Postman (an extremely important tool for testing the /api/search and /api/route endpoints I build before integration).
- Environment: Python Virtual Environment (venv) to manage project libraries in isolation.

2.1.5 Risk Management / Testing Method

Risk	Description	Level	Mitigation Plan
Cost / API Limits	TomTom provides 2,500 free calls/day. If usage exceeds this limit, the service will be suspended.	High	1. Closely monitor the number of daily API calls. 2. Research alternative plans such as using OpenStreetMap data.
Third-party Dependency	Service errors, maintenance, or slow response times from TomTom will paralyze the project's main functionality.	Medium	1. Implement a reasonable timeout/retry mechanism. 2. Clear error handling to notify the user.
Accuracy	Data in HCMC may not be 100% accurate (small alleys, one-way streets, new roads).	Medium	1. Accept as a provider limitation. 2. Apply a filter to only keep results within HCMC, increasing relative accuracy.
Feature Limitations	Limited by the routeTypes TomTom provides. Cannot customize deeply (e.g., "fewer red lights", "fewer lane changes").	Low	1. Adjust the project scope and accept the standard route types (fastest, shortest, balanced, eco).

Table 2.1: Routing risks and mitigations

2.1.6 Timeline

Table 2.2: Routing timeline and milestones

Week	Phase	Main Task (Routing Module)	Objective / Checkpoint
Week 1	A. Design	Requirements Analysis & Scope: - Finalize module scope. - Team meeting to define integration points.	Check: Team agrees on the implementation scope as required.
Week 2	A. Design	Technical Research: - Compare API services (TomTom, Google...). - Begin researching the Folium library.	Check: Decision made to use TomTom API.

Continued on next page

Week	Phase	Main Task (Routing Module)	Objective / Checkpoint
Week 3	A. Design	API Service Design: – Finalize 3 functions (Search, Route, Map). – Share API design (Input/Output) with the team.	Check: Python test script for 3 functions: search, route, map.
Week 4	B. Dev (POC)	Build Proof of Concept (POC): – Start writing Python script. – Integrate TomTom Search & Route.	Check: Script runs and fetches JSON data from TomTom.
Week 5	B. Dev (POC)	Complete POC: – Integrate Folium (WBS 5.3). – Filter “Ho Chi Minh” results.	Check: Code file (.py) works, successfully exports route_map.html.
Week 6	B. Dev	“API-fication” (Part 1): – Install FastAPI, Uvicorn, python-dotenv. – Move search_api logic (WBS 5.1) into /api/search.	Check: Endpoint /api/search (on local) works, returns standard JSON.
Week 7	B. Dev	“API-fication” (Part 2): – Move tomtom_route (WBS 5.2) and visualize_path_on_map (WBS 5.3) logic into /api/route.	Check: Endpoint /api/route (on local) works, returns HTML file.
Week 8	C. Integration	Deploy & Integration: – Deploy the FastAPI/Flask API to a cloud service that supports Python backend, such as Render. – Start supporting the UI/UX Module to embed the interactive map using <code><iframe></code> with the public API URL from Render.	Check: Team has a public API URL. UI can display the map (even with bugs).
Week 9	C. Integration	Error Handling & Integration: – Handle errors (Error Handling 4xx, 5xx). – Support Safety Module & Bus Route Finding System in fetching data.	Check: All other modules can call the API successfully.

Continued on next page

Week	Phase	Main Task (Routing Module)	Objective / Checkpoint
Week 10	D. Deploy	Testing & Optimization: – Fix final integration bugs. – Consider caching (if needed). Finalize documentation.	Check: Functionality is stable in the test environment.
Week 11	D. Deploy	Finalization & Demo: – Ensure API is stable on Production. – Prepare slides & demo.	Check: Ready for the final presentation.

2.2 Recommendation System

2.2.1 Introduction

Public transportation in Ho Chi Minh City is widely available, yet difficult for first-time visitors to navigate efficiently.

While mobile applications such as Google Maps and BusMap provide bus route guidance, they often lack transparency and flexibility in algorithmic design.

This project aims to build an independent system that helps visitors choose the optimal bus route from a starting stop to a destination stop, considering travel time, transfers, and waiting penalties.

However, a key challenge is the lack of open-source and up-to-date bus route data.

To address this, data was collected manually from official city transport sources and combined into structured CSV files, forming the foundational dataset of this project.

2.2.2 Input & Output

Input

- User Input:
 - Start coordinate (latitude, longitude)
 - Destination coordinate (latitude, longitude)
 - MaxWalkingDistance (meter) default = 300 m: the maximum distance the user is willing to walk.
 - MaxFares (VND) default = infinity: the maximum bus fares the user is willing to pay.
 - Routing_mode: cost-efficient / time-efficient
- Dataset Files:
 - `stops.csv` - list of bus stops with coordinates.
 - `routes.csv` - metadata for each bus route (ID, name, fare, active time, spacing penalty).

- `trips.csv` - ordered sequence of stops for each route and distance between them.
- `car_data.json` - used for visualizing bus paths on the map.
- `walk_data.json` - used for visualizing walking paths on the map.

Output

- The optimal route (sequence of bus stops and route IDs) from start to destination.
- The estimated travel distance, estimated best scenario time, estimated worst scenario time (including waiting and transfer penalties).
- The total fares of the trip.
- Visualized route on a map (differentiated each bus transfer).

2.2.3 Work Breakdown

Data Layer

Graph Construction

- Each bus stop is treated as a **node**.
- Adjacent stops on the same route are connected by **directed edges**.
- Edge weights represent **travel distance** (computed with the Haversine formula).
- A **transfer penalty** is added when switching between routes.

Pathfinding Layer

Implements a modified A^* search algorithm that considers:

- Distance between stops.
 - Transfer penalties between routes.
 - Starting penalty for initial waiting time.
- $$\text{cost} = \text{currentcost} + \text{edgeDist}/v + \text{transferPen} + c \times \text{fare} + \text{startingPenalty}.$$
- `currentcost` – accumulated cost to reach the current node.
 - `edgeDist` – distance (in meters) of the current bus segment.
 - `transferPen` (s) – transfer penalty, equal to the average bus spacing / waiting time; set `transfer_pen = 0` if there's no transfer between routes.
 - `v` (m/s) – average bus speed, assumed to be 7 m/s.
 - `c` – constant coefficient that amplifies the importance of bus fare; set `c = 5` if in cost-efficient mode, `c = 0` if in time-efficient mode.
 - `fare` (vnd) – bus fare cost; `fare = 0` if there's no route transfer.

Estimated worst scenario time = TotalCost - TotalFares

Estimated best scenario time = TotalCost - TotalFares - TotalTransferPenalties

TotalDistance = estimated best scenario time $\times v$ + TotalWalkingDistance

The algorithm can compare multiple potential starting stops and select the route with the lowest total cost.

Realistic Modeling Features

- **Real-time input:** Automatically remove bus routes not on active time.
- **Transfer Penalty:** Adds time when changing from one route to another, reflecting bus waiting time.
- **Starting Penalty:** Models the time a visitor might wait for the first bus to arrive.
- **Heuristic Function:** Uses geographical distance (via the Haversine formula) to guide the A^* search efficiently.

2.2.4 Requirements

Hardware

- **Development Machine:** 1 Personal Laptop (Windows, macOS, or Linux) with an internet connection for programming and API testing.
- **Deployment Server:** Cloud hosting service (such as Render) that supports Python backend applications.

This platform will host the FastAPI/Flask internal API and provide a public endpoint for integration with other modules.

Software

- **Language:** Python (3.10+ recommended) as the primary implementation language.
- **Data Handling:** `pandas` and Python's `csv` module for reading, validating, and transforming bus stop and route data from CSV files (e.g., `stops.csv`, `routes.csv`, `trips.csv`).
- **Graph Algorithms:** Custom implementations of A^* and Dijkstra with a pluggable cost function that supports penalties for waiting time, number of transfers, and walking distance; must return the path (sequence of stop IDs and route IDs) and aggregate cost/time metrics.
- **Math & Geo Utilities:** Standard `math` package and a Haversine formula utility for great-circle distance between coordinates; define a small *epsilon* tolerance for coordinate comparisons.
- **Visualization:** `folium` to render routes and stops on an interactive web map and export to a standalone HTML file for demos/presentations.
- **Documentation:** A Markdown report (`report.md`) describing dataset schemas, algorithm design/parameters, CLI usage examples, and known limitations.

2.2.5 Timeline

Week	Task	Description
Week 1 – Week 2	Research and Problem Definition	Study existing transport apps, define goals and system constraints. Identify the absence of open-source bus data.
Week 3 – Week 4	Data Collection and Cleaning	Manually collect bus stop, route, and trip data from official or public sources. Convert them into standardized CSV format (<code>stops.csv</code> , <code>routes.csv</code> , <code>trips.csv</code>).
Week 5 – Week 6	Graph Construction	Implement logic to parse CSV files, create nodes and weighted edges, and structure the graph for routing (directed edges between adjacent stops, Haversine-based weights, transfer/start penalties).
Week 7 – Week 8	Algorithm Development	Implement Dijkstra and A^* algorithms with transfer and spacing penalties. Validate correctness with sample cases and unit tests.
Week 9 – Week 10	Testing and Optimization	Test with multiple start/destination combinations. Fine-tune penalties and heuristic to ensure realistic results and acceptable performance.
Future Work	Integration and Visualization	Build a simple UI or map visualization to display the optimal bus route and estimated travel time to users (folium/OSM integration, optional lightweight web frontend).

Table 2.3: Recommendation system timeline and milestones

2.3 Safety & Monitoring

2.3.1 Introduction

HCMC endless traffic congestions especially in rush hour cannot be solved easily as the city is growing rapidly in both population and vehicles.

To help commuters avoid traffic jams, we propose a traffic-aware routing feature that leverages real-time traffic camera feeds to analyze current traffic conditions and suggest alternative route.

By integrating computer vision techniques with graph-based routing algorithms, this feature aims to enhance urban mobility and reduce travel times for users navigating through Ho Chi Minh City.

Impact

- Inefficient planning and delays.
- Poor commuter satisfaction.

2.3.2 Input & Output

Input

- Start location.
- End location.

Output

- Coloured route according to the current traffic flow.
- Suggest one alternative less crowded route (if not possible, display a notice).
- A cute note for users.

2.3.3 Work Breakdown

Data & Telemetry Pipeline Ingestion

- Pull JPG snapshots every 30 minutes from official HCMC traffic camera endpoints (city traffic portal).
- De-duplicate by perceptual/image hash to avoid storing identical frames.
- Bind metadata: `camera_id`, `segment_id`, GPS, bearing, lane schema, timestamp.
- Persist raw frames + minimal JSON into object storage (Bronze layer).

Curation Layers

- Bronze: raw frames + minimal JSON (timestamp, `camera_id`).
- Silver: per-frame detections, tracks, lane associations, calibration records, COCO annotations (labeled subset).
- Gold: per-segment time series every 1 hour: density (veh/km/lane), occupancy, queue tail, space-mean speed, incident flags.

Labeling Strategy

- Seed with 20–30k annotated frames (COCO boxes; classes: four-wheel vehicles, motorbike).
- Active learning to select “hard” frames (night rain, glare, heavy occlusion); add 1–2k frames per corridor during pilot.

Per-camera Calibration & Geo-projection

- Use road corners to compute homographies.
- Back-project pixel distances to meters for speed/queue length estimation (standard monocular stationary-camera procedure).

Perception Stack

Pre-processing

- Defog/dehaze and gamma correction as first-stage normalization (stabilize detectors across dusk/night and haze).
- Legacy self-adaptive background subtraction with dynamic thresholds as secondary fallback for heavy occlusion/static cams.

Detection

- YOLO family (anchor-free head, e.g., YOLOv11-S); train with augmentation (mosaic, copy-paste), focal loss, finer grids for motorbikes.

Tracking & Lane Association

- ByteTrack or OC-SORT at 2–5 FPS for ID-consistent tracks.
- Per-track lane assignment via calibrated lane polygons.
- Compute space-mean speed from track displacement in world coordinates.

Segment-level Signals (aggregation window $\Delta t = 2\text{--}4$ min)

- Density (veh/km/lane), occupancy, speed percentiles, stop-and-go wave indicators; aggregate into Gold time series.

Short-horizon Traffic Prediction (5–30 min) Problem Framing

- Predict per-segment congestion class (R/Y/G) and/or travel time / mean speed at 5, 10, 20, 30 min ahead.
- Inputs: recent Gold signals + neighbor segment features.

Models

- Champion spatio-temporal GNN: DCRNN / STGCN / GraphWaveNet over road graph (nodes = segments, edges = adjacency/turns).

Feature Set & Horizons

- Sliding windows: last 30–90 min at 2–4 min resolution; lags, moving averages, trend features.

- Exogenous: hour/day/week, holiday indicator, weather (rain intensity, temperature), seasonality.

Losses & Metrics

- Regression (speed/ETA): RMSE, MAE, R^2 .
- Classification (R/Y/G): macro-F1, precision, recall; horizon-specific evaluation (≤ 30 min is primary use-case).

Time-dependent Routing

Graph & Costs

- Build time-dependent directed graph from OSM + city topology.
- Edge cost = predicted travel time $\times$ congestion penalty multiplier (e.g., Red 1.35, Yellow 1.15) + uncertainty surcharge when predictor confidence is low.

Search

- A* with landmark heuristics; do not exceed shortest-path ETA by $>10\%$ unless red-jam risk predicted in next 15 min; avoid flooded links under weather alerts.

UX

- Show route colored by predicted flow.
- Provide one named alternative with rationale: “Queue forming at Nguyễn Hữu Cánh in 10 min; taking Điện Biên Phủ saves 7–10 min.”
- Cute note (copy layer): “Sky is moody, roads are not—swing right to avoid the jam ahead.”

Privacy & Safety

- No PII retention; mask plates/faces if visible.
- Edge redaction before storage; compliance with local data policies.

Ambiguities & Limitations

- Motorbike occlusions in dense rain: under-count risk; mitigate via temporal smoothing + uncertainty flags (Medium limitation).
- Calibration drift (camera bumps; road works): re-fit homographies monthly or on drift alerts; residual errors affect speed more than counts (Low–Medium).
- Weather data latency/coverage: delayed granular rain data degrades prediction during storms; cache nearby station feeds and backfill (Low–Medium).

2.3.4 Requirements

Hardware

- **Edge node (per camera or small group):** CPU with 8–16 GB RAM; (optional GPU). Local SSD for short-term buffering (~24–48 h) and logs.
- **Central node (training/aggregation):** Multi-core CPU, CUDA-capable GPU for fine-tuning detectors/GNNs; network-attached storage for Silver/Gold tables and artifacts.
- **Network:** Stable uplink from cameras or RTSP gateway; resilience via buffering and auto-retry on loss.

Software

- **Perception:** YOLO family detector (YOLOv8/YOLOv11-S), ByteTrack/OC-SORT tracker, homography-based calibration utilities.
- **Data pipeline:** Services to write Bronze/Silver/Gold layers; time-window aggregation (2–4 min); REST/WebSocket publisher.
- **Prediction:** Spatio-temporal GNN framework (DCRNN/STGCN/GraphWaveNet) and a tree-based fallback (RF/GBT).
- **Dashboard:** Web UI for live map, history, and metrics; role-based access; configuration for junction onboarding.

Functional Requirements (FR)

ID	Requirement	Description
FR1	Detector Setup	YOLOv11 model with vehicle classes; junction fine-tuning.
FR2	Tracking & Counting	Multi-object tracker + virtual lines per lane/movement; de-dup via track IDs.
FR3	Class-wise Flow	Aggregate counts per 10–15 s (configurable) by class & movement.
FR4	Camera Calibration	Homography mapping pixels → lanes/movements.
FR5	Graph Builder	Construct directed graph (junctions/edges) with rolling features.
FR6	GNN Predictor	Short-term congestion per edge using recent flows & exogenous signals.
FR7	Alerting	Map scores to R/Y/G with thresholds + hysteresis; publish via REST/WebSocket.
FR8	Dashboard	Live map with colored segments; history & metrics view.
FR9	Evaluation	mAP (detector), MAE/MAPE (counts), P/R/F1 (alerts); periodic reports.

Non-Functional Requirements (NFR)

ID	Requirement	Description
NFR1	Latency	Edge detection+count $\leq 300\text{--}600$ ms/frame; end-to-end alert $\leq 3\text{--}5$ s.
NFR2	Accuracy	Counting MAE $\leq 10\%$ on audited windows; Alert F1 ≥ 0.85 (initial).
NFR3	Robustness	Operate across day/night & rain; track degradation.
NFR4	Privacy	No faces/plates stored; only aggregates persist.
NFR5	Reliability	99% pipeline uptime; auto-restart & buffering on network loss.
NFR6	Maintainability	Modular services; config-driven junction onboarding.

2.3.5 Risk Management / Testing Method

- **Perception (offline)** mAP@50 / @50–95 by class; IDF1/HOTA for tracking; speed MAE/MAPE vs radar/loop ground truth.
- **Prediction (offline)** Cross-validated MAE/RMSE (speed/ETA); macro-F1/precision/recall (R/Y/G); analyze by horizon (5/10/20/30 min) and weather bins.
- **Shadow Mode (online)** Compare predicted vs realized congestion/ETA for 4–6 weeks; calibrate alert thresholds to reach $\geq 80\%$ precision before public alerts.

2.3.6 Timeline

Week	Task	Description / Exit criteria
1	Project Setup & Governance	Repo, CI, coding style; success metrics (mAP/IDF1, MAE/F1), data policy; camera list & schema freeze. Deliverable: 1–2 pp scope & risk register.
2	Ingestion Prototype (Bronze)	Pull JPG snapshots from HCMC endpoints every 30 min; perceptual hash de-dup; bind metadata (camera_id, GPS, bearing). Deliverable: 24 h Bronze sample.
3	Label Seed & Calibration Kit	Curate 20–30k COCO boxes (car/motorbike); active-learning sampler; homography tool (road corners). Deliverable: 5 calibrated cams + seed set.
4	Detector Baseline	Train/tune YOLOv8/YOLOv11-S with augmentation; night/rain sanity checks. Exit: mAP@50 baseline achieved; error notebook for hard scenes.
5	Tracking & Counting	ByteTrack/OC-SORT at 2–5 FPS; virtual lines per lane/movement; de-dup via track IDs. Exit: MAE $\leq 10\text{--}15\%$ on audited windows.
6	Silver/Gold Pipeline & Metrics	Aggregate $\Delta t = 2\text{--}4$ min to density/occupancy/speed; persist Silver/Gold; dashboard skeleton. Exit: 1 week of Gold time series.
7	Road Graph & Features	Build directed graph (OSM + topology); engineer features/lags; create train/val splits; tree baseline (RF/GBT) for ETA/speed. Exit: offline MAE/RMSE baseline.
8	Spatio-temporal GNN	Train DCRNN/STGCN/GraphWaveNet for 5/10/20/30-min horizons; calibrate uncertainty; map to R/Y/G with hysteresis. Exit: macro-F1 target for ≤ 30 min.
9	Time-dependent Routing (A*)	Edge cost = predicted travel time $\times$ congestion penalty + uncertainty surcharge; one named alternative route + rationale; flood avoidance hook. Exit: prototype UI coloring.
10	Shadow Mode & Tuning	Live compare predicted vs realized congestion/ETA; threshold calibration to reach $\geq 80\%$ precision before alerts; privacy audit (plate/face masking).
11	Hardening & Demo	Latency profiling (edge $\leq 300\text{--}600$ ms/frame; E2E alert $\leq 3\text{--}5$ s); reliability tests (auto-retry/buffering); docs, ops runbook, demo with “cute note”.

Table 2.4: Safety & Monitoring module: 11-week timeline and milestones

2.4 Camera Ingestion Pipeline

2.4.1 Introduction

Project purpose

Collect, normalize, and serve legally compliant real-time traffic camera frames to support downstream modules (object detection, flow estimation, congestion scoring, safety alerts). The pipeline must deliver low-latency, robust, and auditable frame ingestion while protecting privacy and meeting retention policies.

Scope

- Continuous ingestion from many heterogeneous camera sources (varying resolution, FPS, protocols).
- Per-camera preprocessing (ROI, normalization), calibration for geometric reasoning, short-term storage for debugging, and health/metrics dashboards.
- API access for downstream consumers and operators, plus compliance documentation and privacy protections.

Key goals

- Reliable frame delivery (target freshness SLA).
- Privacy and legal compliance for each feed.
- Observability and easy onboarding of new cameras.

2.4.2 Input & Output

Inputs

- Camera feed endpoints (RTSP/HTTP/HLS/etc.) and credentials.
- Camera metadata: `camera_id`, URL, GPS, orientation, vendor, resolution, nominal FPS, tilt/azimuth.
- Permission/terms-of-use / proof of right to ingest.
- Per-camera ROI masks and calibration images (for homography).
- Central configuration (sampling rate, retention policy, alert thresholds).

Outputs

- Sampled frames (JPEG/PNG) and short clips retained for 24–72 hours in object storage.
- Per-frame metadata records (timestamp, `camera_id`, `seq_no`, `ingest_latency`, `checksum`) in Postgres.

- Calibration artifacts (homography matrices, ROI masks) stored alongside camera metadata.
- Time-series metrics (frame freshness, fetch rate, errors, latency) in TimescaleDB/InfluxDB and Grafana dashboards.
- HTTP/REST API endpoints returning latest frames, frame metadata, and camera health.
- Audit & compliance report (camera permissions, retention logs, redaction events).

2.4.3 Work Breakdown

Camera Onboarding & Registry

- Build camera registry schema (Postgres): metadata, legal status, sampling policy.
- CLI / UI to add/update cameras and upload ROI/calibration images.

Scraper & Fetcher

- Implement resilient fetcher supporting RTSP/HLS/HTTP with backoff, retries, rate limiting.
- Health checks, circuit breakers, and opt-in buffering for high-latency feeds.

Frame Extraction & Sampling

- FFmpeg integration to sample frames at configurable rates (default 1 fps) and normalize containers.
- Edge cases handling (variable FPS, partial frames).

Preprocessing & Calibration

- Image normalization (resize, color/contrast adjustment) using OpenCV.
- Apply ROI masks, blur redaction zones if required.
- Compute / store homography for selected cameras for downstream speed/trajectory estimation.

Storage & Retention

- Short-term frame storage: S3/MinIO with lifecycle rules (24–72 h).
- Persistent metadata in Postgres and time-series metrics in TimescaleDB/InfluxDB.

API & Access

- FastAPI endpoints: register/list cameras, latest frame, health, metrics.
- AuthN/AuthZ for operators and downstream services.

Monitoring, Alerts & Observability

- Export metrics (Prometheus) and dashboards (Grafana) for frame freshness, success rate, latency.
- Alert rules (stale frames, high error rate, permission expiry).

Compliance & Privacy

- Legal review workflow per camera; store permission evidence.
- Automatic face/license-plate blurring or ROI redaction when required.
- Retention enforcement and audit logs.

2.4.4 Requirements

Functional Requirements

1. Register and manage camera metadata and legal status.
2. Continuously fetch and sample frames from registered cameras.
3. Provide per-camera ROI masking and optional redaction.
4. Expose API to retrieve latest frames and camera health.
5. Store frames for configurable short retention window and metadata indefinitely.

Non-Functional Requirements

- **Latency:** end-to-end frame availability within target (e.g., < 2 s after sample time for on-network cameras).
- **Scalability:** support N concurrent camera feeds (design target e.g., 1k) with horizontal scaling.
- **Reliability:** per-camera success rate target (e.g., $> 99\%$ over rolling 24 h).
- **Security:** encrypted in-transit and at-rest; credentials stored safely.
- **Observability:** metrics for freshness, errors, and throughput; logs for forensic audits.

Legal & Compliance

- Track and store permission documents for each camera feed.
- Implement automatic redaction if terms require (faces, plates) and log redaction events.
- Enforce retention policy and provide exportable compliance evidence.

Deployment & Ops

- Containerized services (Docker) with CI/CD (GitHub Actions).
- Infrastructure as code (Terraform) for environments and S3/MinIO provisioning.
- Runbooks for onboarding and incident response.

2.4.5 Risk Management / Testing Method

Major Risks & Mitigations

- **Feed instability / network latency:** implement retry/backoff, local buffering, and fallback sampling frequency.
- **Heterogeneous stream formats / codecs:** standardize via FFmpeg transcoding layer and build format compatibility tests.
- **Privacy / legal exposure:** require documented permissions before enabling a camera; automatic redaction; legal review gate.
- **Scaling limits under high concurrency:** autoscaling policies, connection pooling, and rate limiting per-camera.
- **Poor image quality (weather, night):** flag low-quality frames and route to quality-aware downstream models or human review.

Testing Strategy

1. **Unit tests:** parser, registry CRUD, config validation.
2. **Integration tests:** end-to-end sampling from a synthetic stream, storage round-trip, metadata writes.
3. **Contract tests:** API schema/response shape for downstream consumers.
4. **Load & Stress tests:** simulate target concurrent feeds and max throughput (sampled frame rate $\times N$ cameras).
5. **Fault injection / Chaos testing:** simulate network drops, FFmpeg failure, permissions revocation to verify graceful recovery.
6. **Security & privacy tests:** verify redaction effectiveness (face/plate blur), permissions enforcement, token expiry.
7. **Acceptance tests / E2E:** run a validation scenario covering registration $\rightarrow$ sampling $\rightarrow$ API retrieval $\rightarrow$ metric logging.

Acceptance Criteria

- Freshness SLA met for 95%+ of feeds in baseline environment.
- Error rate below agreed threshold (e.g., < 1% critical failures).
- Permissions recorded for every active camera; redaction applied where required.
- Monitoring dashboards and alerts firing for defined conditions.

2.4.6 Timeline

Table 2.5: Camera Ingestion Pipeline timeline and deliverables

Week	Focus	Output / Deliverables
Week 0	Setup & Legal	Camera inventory, permissions review, sampling policy & test plan
Week 1–2	Pipeline Core	Scraper, FFmpeg extractor, camera registry, basic health monitoring
Week 3–4	Preprocessing	Frame normalization, ROI masking, redaction prototype
Week 5	Calibration	Homography / perspective calibration for selected cameras
Week 6	Metrics & Logging	Frame freshness and uptime metrics stored in time-series DB
Week 7	API Integration	Endpoints for latest frame, camera metadata, and health
Week 8	Privacy Controls	Blurring/masking + retention enforcement implementation
Week 9	Dashboards	Grafana monitoring dashboard and alert thresholds
Week 10	E2E Testing	Full validation of pipeline stability, latency, and failure modes
Week 11	Documentation	Runbooks, onboarding guide, compliance evidence packaging

2.5 Database

2.5.1 Introduction

Persist, index, and serve all mobility data for routing, congestion prediction, and UX layers. Normalize heterogeneous sources (Overpass/OSM, GTFS-like CSVs, camera-derived signals) into time-aligned, queryable stores. Provide stable contracts (SQL/Parquet views + APIs) to upstream ML (GNN/TBMs) and downstream services (A* router, map UI).

Scope (what it does not own)

- No model training/inference logic (served by ML service).
- No image processing (YOLO/ByteTrack) — Database only stores outputs & meta-data.

2.5.2 Input & Output

Inbound data (batch/stream)

- **Overpass/OSM snapshots (JSON)**: roads, nodes, ways; Python fetchers already in place.
- **City cameras** → per-frame detections/tracks → per-segment aggregates (density, queue length, speed proxies).
- **GTFS-like CSVs**: `routes.csv`, `stops.csv`, `trips.csv` (for stop proximity, segment naming, UX overlays).
- **Weather/seasonality features** (hour, weekday, rain flags) for ML enrichment.

Canonical schemas (storage contracts)

Bronze (immutable, as-landed)

- `raw_overpass` (JSON blob + bbox, `fetched_at`).
- `raw_frames` (uri, camera_id, ts, img_hash).
- `raw_detections` (frame_id, class, bbox, score).

Silver (cleaned & derived)

- `lane_calibration` (camera_id, lane_id, homography, `updated_at`).
- `tracks` (track_id, camera_id, lane_id, ts, world_x/world_y, speed_est, is_stalled).
- `segments` (segment_id, geom, speed_limit, lanes, osm_way_ids).

Gold (time series for ML & routing)

- `segment_signals_2m` (ts, segment_id, density_veh_per_km_lane, occupancy, speed_p50, speed_p85, queue_tail_m, incident_flag, weather_bins).
- `congestion_labels_2m` (ts, segment_id, class ∈ {G, Y, R}, label_method, confidence).
- `travel_time_edge_ETA_2m` (ts, edge_id, eta_s, eta_conf, horizon_min ∈ {5, 10, 20, 30}).

Outbound interfaces

SQL/Views for ML: sliding-window tensors (RNN/GNN ready), normalized features (MinMax, one-hot).

HTTP/gRPC (internal):

- GET /db/segments/{id}/signals?from=&to=
- GET /db/edges/eta?at=...&horizon=...
- POST /db/congestion_labels:write (for online labels / human QA)

2.5.3 Work Breakdown

Data Modeling & Storage

Define PostGIS schemas for segments, edges, stops, routes; Parquet for high-volume time series (signals, ETA). Partition by date (daily) + `segment_id` bucketing; ZSTD compression.

Ingestion Pipelines

Overpass puller → `raw_overpass`; Camera snapshot harvester → hash-dedupe → `raw_frames`; detection/tracking appends to `raw_detections/tracks`; CSV loaders for `routes.csv`, `stops.csv`, `trips.csv`.

Transformation

Build segments from OSM ways (respect oneway, service exclusions). Aggregate tracks → `segment_signals_2m` with calibrated lane polygons. Join exogenous features (weather, seasonality).

Serving

Materialized views for ML (sliding windows: last 30–90 min) and router (time-dependent edge costs). Low-latency KV (Redis/KeyDB) cache for “current” 0–30 min horizons.

Governance & Ops

Data retention policy (Bronze 90 d; Silver/Gold 365 d rolling). CDC/versioning for calibration & segment topology. Backfills + schema migration playbooks.

2.5.4 Requirements

Functional

- Store ≥ 180 days of `segment_signals_2m` for model retraining and seasonality.

- Provide ≤ 300 ms P95 reads for latest 30-minute window per corridor to serve A* routing.
- Offer labeled congestion classes (R/Y/G) + confidence for alerts (DT/RF fallback proven strong).

Non-functional

- Reliability: 99.5% monthly availability for reads; durable storage (daily backups).
- Latency: ingest $\rightarrow$ Gold availability ≤ 90 s after frame capture.
- Security/Privacy: no PII; plate/face hashes never stored.

2.5.5 Risk Management / Testing Method

Key risks & mitigations

- Camera downtime/data gaps $\rightarrow$ impute from neighbors + seasonality; flag low-confidence.
- Night/rain occlusion $\rightarrow$ temporal smoothing; fallback detectors.
- Calibration drift $\rightarrow$ version homographies, CDC rollback.
- OSM mismatch $\rightarrow$ maintain override layer + periodic diffs.

Testing strategy

- Schema & contract tests: migration dry-runs, backward-compatibility.
- Ingest fidelity tests: checksum & row-count parity.
- Feature quality tests: drift monitors, missingness dashboards.
- Latency SLO tests: synthetic load (1k segments).
- Label QA: macro-F1 tracking (DT/RF baseline ≥ 0.90).

2.5.6 Timeline

Week	Focus	Deliverables	Certainty
1–2	Foundations	ERD, PostGIS/Parquet setup, Bronze/Silver/Gold scaffolding, CSV loaders	High
3–4	Camera → Signals path	<code>raw_frames/raw_detections</code> landed, aggregation → <code>segment_signals_2m</code> , calibration versioning	Medium-High
5	Gold & Serving	Gold views, ML exports, <code>travel_time_edge_ETA_2m</code> , Redis cache	Medium
6	QA & Hardening	Quality monitors, backfills, retention/backup, load & label QA loop	Medium

2.6 UI/UX Platform

2.6.1 Introduction

Urban travelers today face a fragmented mobility experience — switching between multiple apps for bus schedules, ride-hailing, and walking navigation. This results in:

- Inefficient trip planning (balancing time, cost, and distance).
- Lack of real-time actionable insights (traffic, ETA drift).
- User frustration and cognitive fatigue.

UI/UX Goal Design a unified, intuitive, and responsive interface that allows users to:

- Plan, book, and monitor their trips end-to-end in one platform.
- Receive real-time recommendations with clear trade-offs.
- Navigate safely with contextual alerts and visual clarity.

2.6.2 Input & Output

Category	Description
Input	Start & destination points, preferred transport modes, live GPS data, transit APIs, local build API for object detection, and user preferences.
Output	Optimized travel itineraries (Fastest/Normal), ETA & cost estimates, congestion alerts, and real-time route updates.

System Flow

1. User enters trip details.
2. System fetches live transport & traffic data & camera data.
3. UI displays top route options.
4. In-trip recommender updates dynamically via GPS and APIs.
5. Alerts and rerouting offered in case of congestion.

2.6.3 Work Breakdown

extbfModule	Description	Deliverables
A. User Research & Persona	Identify user pain points via surveys and interviews.	Persona profiles, user journey maps
B. Information Architecture	Design navigation across Pre-trip, In-trip, and Alerting modules.	Sitemap, navigation flow
C. Wireframe (Low-Fidelity)	Build layout skeletons for key screens (Trip Planner, Tracking).	Low-fi wireframes
D. Prototype (High-Fidelity)	Apply colors, typography, icons, and animations.	Interactive Figma prototype
E. UI Implementation	Convert designs to React/Next.js frontend with live API data.	Responsive working UI
F. UX Testing & Iteration	Conduct usability tests and refine UI based on feedback.	UX test report, iteration summary

2.6.4 Requirements

Functional Requirements

- Allow users to input and edit trip details (start/end points, mode).
- Display optimized routes with ETA, cost, and traffic levels.
- Provide real-time updates and one-tap reroute options.
- Deliver congestion & safety alerts.
- Ensure accessibility compliance (font size $\geq 14\text{px}$, contrast ratio $\geq 4.5 : 1$).

Non-Functional Requirements

Aspect	Specification
Usability	$\geq 85\%$ success rate in usability tests
Performance	Response latency $\leq 5\text{ s}$ (p95)
Reliability	$\geq 99.5\%$ crash-free sessions
Accessibility	WCAG 2.1 compliance
Consistency	Unified icons, color scheme, and feedback states

Tools & Frameworks

Purpose	Tools / Frameworks
UI Design	Figma, Adobe XD, Illustrator
Prototyping	Figma Prototype, Framer, InVision
Frontend	React.js, Next.js, Tailwind CSS, shadcn/ui
UX Testing	Maze, Hotjar, Google Analytics
Collaboration	Notion, Trello, Slack, GitHub

2.6.5 Risk Management / Testing Method

Risk Analysis

Risk	Impact	Mitigation Strategy
API data inconsistency (ETA drift)	Medium	Cache fallback data; re-request on failure.
Overwhelming UI complexity	High	Simplify actions: ≤ 3 taps for core functions.
Accessibility issues	High	Use accessible color palette and test with screen readers.
Integration delays	Medium	Align weekly via shared Figma + GitHub sync.
Low participation in usability testing	Low	Utilize online testing via Maze/Hotjar.

Testing Methods

- **Usability Testing:** Evaluate ease of use and completion rate of key actions.
- **A/B Testing:** Compare alternative UI layouts for efficiency.
- **Heuristic Evaluation:** Check compliance with UX principles.
- **Performance Testing:** Measure app response time and smoothness.
- **User Satisfaction Survey:** Target CSAT $\geq 4.2/5$.

2.6.6 Timeline

Week	Phase	Key Activities	Deliverables
Week 1–2	Research & Persona	Conduct surveys/interviews and map user journeys.	Persona & journey map
Week 3	Information Architecture	Design sitemap and define module structure.	Sitemap & flow diagram
Week 4–5	Wireframing	Create low-fidelity wireframes for all key screens.	Approved wireframes
Week 6–7	High-Fidelity Design	Apply brand system, icons, and UI interactions.	Interactive prototype
Week 8	UI Implementation	Develop and integrate React/Next.js frontend.	Working UI with API link
Week 9–10	UX Testing & Iteration	Conduct usability tests, analyze feedback.	UX Report + Refined Design
Week 11	Final Presentation	Prepare final demo and design deck.	Live demo & presentation materials